

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ**

**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ВЯТСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»**

Институт математики и информационных систем

Факультет автоматики и вычислительной техники

Кафедра электронных вычислительных машин

Отчёт по лабораторной работе №3
по дисциплине

«Объектно-ориентированное
программирование»

Выполнил студент гр. ИВТб-2302-05-00

_____/Матешко В.В./
(Подпись)

Проверил старший преподаватель

_____/Шмакова Н.А./
(Подпись)

Киров
2026

Цель работы

Разработать веб-приложение, используя классы и особенности Java, демонстрирующее применение ООП и работу через Hibernate.

Задание

На основе предметной области разработать приложение на Java с веб-интерфейсом. Данные об объектах предметной области хранить в PostgreSQL. Для работы с БД использовать Hibernate, а для базовых CRUD-операций использовать `CrudRepository`.

Требования к реализации

1. Использование языка программирования Java;
2. Использование СУБД PostgreSQL;
3. Использование сборщика Maven;
4. Использование Hibernate для подключения к БД;
5. **Требование к БД:** не менее 3-х связанных таблиц в 3-ей нормальной форме. Обязательно хотя бы одна связь не по наследованию;
6. Веб-интерфейс;

Ход работы

Предметная область: универсальные API-подключения (VK и Telegram).

Описание классов предметной области

Базовый абстрактный класс Api:

- Тип класса и модификатор доступа: `public abstract class Api`;
- Поля:
 - `private Long id` — первичный ключ;
 - `protected String name` — название API-подключения;
 - `protected String baseUrl` — базовый URL;
 - `protected Organization organization` — ссылка на организацию (many-to-one).
- Методы:
 - `public String initialize()` — возвращает строку "Initializing API...";
 - `public String fetchData(String command)` — базовая реализация, возвращает строку "Загрузка данных..."; в потомках может переопределяться для предметной команды.
- Конструкторы:
 - `protected Api()` — пустой конструктор для Hibernate;
 - `protected Api(String name, String baseUrl)` — инициализирует родительские поля `name` и `baseUrl`.
- Геттеры/сеттеры:
 - Для полей `id`, `name`, `baseUrl`, `organization` используются Lombok-аннотации `@Getter` и `@Setter`.

Класс VkApi extends Api:

- Тип класса и модификатор доступа: `public class VkApi extends Api`;
- Поля:
 - `private String version` — версия VK API;
 - `private String token` — токен доступа;
 - `private List<VkPost> posts` — связанные посты (one-to-many).
- Методы:
 - `public String initialize()` — наследует родительское поведение и может вызываться через API;
 - `public String fetchData(String command)` — обобщённый вызов команды для демонстрации полиморфного доступа;
 - `public List<String> getText()` — возвращает список текстов постов;
 - `public List<Integer> getLike()` — возвращает список значений лайков.
- Конструкторы:

- `protected VkApi()` — пустой конструктор для Hibernate;
- `public VkApi(String version, String token)` — создаёт VK API с явными `version` и `token`, а `name/baseUrl` задаёт как "VK" и "https://api.vk.com/method";
- `public VkApi(String token)` — создаёт VK API с токеном и версией по умолчанию "5.199".
- Геттеры/сеттеры:
 - Для полей `version`, `token`, `posts` используются Lombok-аннотации `@Getter` и `@Setter`.

Класс `TgApi` extends `Api`:

- Тип класса и модификатор доступа: `public class TgApi extends Api`;
- Поля:
 - `private String botToken` — токен бота;
 - `private long chatId` — идентификатор чата;
 - `private List<TgPayload> payloads` — связанные данные сообщений (one-to-many).
- Методы:
 - `public String initialize()` — наследует родительское поведение и может вызываться через API;
 - `public String fetchData(String command)` — обобщённый вызов команды для демонстрации полиморфного доступа;
 - `public List<String> sendMessage()` — возвращает тестовые данные отправки сообщения;
 - `public List<String> sendPhoto()` — возвращает тестовые данные отправки фото.
- Конструкторы:
 - `protected TgApi()` — пустой конструктор для Hibernate;
 - `public TgApi(String botToken, long chatId)` — создаёт Telegram API с токеном и чатом, а `name/baseUrl` задаёт как "TG" и "https://api.telegram.org/bot";
 - `public TgApi(String botToken)` — создаёт Telegram API только с токеном, `chatId` устанавливается в 0L.
- Геттеры/сеттеры:
 - Для полей `botToken`, `chatId`, `payloads` используются Lombok-аннотации `@Getter` и `@Setter`.

Класс `VkPost`:

- Тип класса и модификатор доступа: `public class VkPost`;
- Поля:
 - `private Long id` — идентификатор поста;
 - `private String text` — текст поста;
 - `private int likes` — количество лайков;
 - `private VkApi vkApi` — ссылка на родительский VK API (many-to-one).
- Конструкторы:
 - `protected VkPost()` — пустой конструктор для Hibernate;

- `public VkPost(String text, int likes, VkApi vkApi)` — создаёт пост и привязывает его к VK API.

Класс `TgPayload`:

- Тип класса и модификатор доступа: `public class TgPayload`;
- Поля:
 - `private Long id` — идентификатор записи данных;
 - `private String payloadType` — тип данных (`message/photo`);
 - `private String payloadData` — содержимое данных;
 - `private TgApi tgApi` — ссылка на родительский TG API (`many-to-one`).
- Конструкторы:
 - `protected TgPayload()` — пустой конструктор для Hibernate;
 - `public TgPayload(String payloadType, String payloadData, TgApi tgApi)` — создаёт запись данных и привязывает её к TG API.

Класс `Organization`:

- Тип класса и модификатор доступа: `public class Organization`;
- Поля:
 - `private Long id` — идентификатор организации;
 - `private String name` — название организации;
 - `private List<Api> apis` — список связанных API-записей (`one-to-many`).
- Конструкторы:
 - `protected Organization()` — пустой конструктор для Hibernate;
 - `public Organization(String name)` — создаёт организацию с заданным именем.

ООП в проекте

- Инкапсуляция: поля сущностей скрыты, доступ к ним осуществляется через геттеры/сеттеры (Lombok `@Getter/@Setter`);
- Наследование: `VkApi` и `TgApi` наследуются от базового класса `Api`;
- Полиморфизм: вызов родительских методов `initialize()` и `fetchData(...)` выполняется для разных наследников через единый интерфейс `API`.

Описание API:

VK API:

- GET `/api/vkapis` — получить список всех VK API-записей;
- GET `/api/vkapis/{id}` — получить одну VK API-запись по `id`;
- POST `/api/vkapis/full` — создать VK API через конструктор (`version, token`);
- POST `/api/vkapis/token` — создать VK API через конструктор (`token`);

- `POST /api/vkapis/default` — создать VK API со значениями по умолчанию;
- `DELETE /api/vkapis/{id}` — удалить VK API-запись по id;
- `GET /api/vkapis/{id}/initialize` — вызвать родительский метод `initialize()`;
- `GET /api/vkapis/{id}/fetchdata/{command}` — вызвать родительский метод `fetchData(command)`;
- `GET /api/vkapis/{id}/run/{command}` — вызвать предметные методы VK (`getText`, `getLike`).

TG API:

- `GET /api/tgapis` — получить список всех TG API-записей;
- `GET /api/tgapis/{id}` — получить одну TG API-запись по id;
- `POST /api/tgapis/full` — создать TG API через конструктор (`botToken`, `chatId`);
- `POST /api/tgapis/token` — создать TG API через конструктор (`botToken`);
- `POST /api/tgapis/chat` — создать TG API по `chatId`, при этом `botToken` выставляется в "NULL";
- `DELETE /api/tgapis/{id}` — удалить TG API-запись по id;
- `GET /api/tgapis/{id}/initialize` — вызвать родительский метод `initialize()`;
- `GET /api/tgapis/{id}/fetchdata/{command}` — вызвать родительский метод `fetchData(command)`;
- `GET /api/tgapis/{id}/run/{command}` — вызвать предметные методы TG (`sendMessage`, `sendPhoto`).

Описание графического интерфейса

Интерфейс реализован на React и включает:

- формы добавления VK и TG API записей;
- форма редактирования записей по ID;
- кнопки удаления записей;
- блок вызова методов предметной области (`Initialize`, `FetchData`, `run(command)`) через выпадающие списки;
- блок отображения результата вызова методов и ошибок;
- кнопку обновления списков записей из БД.

ERD диаграмма

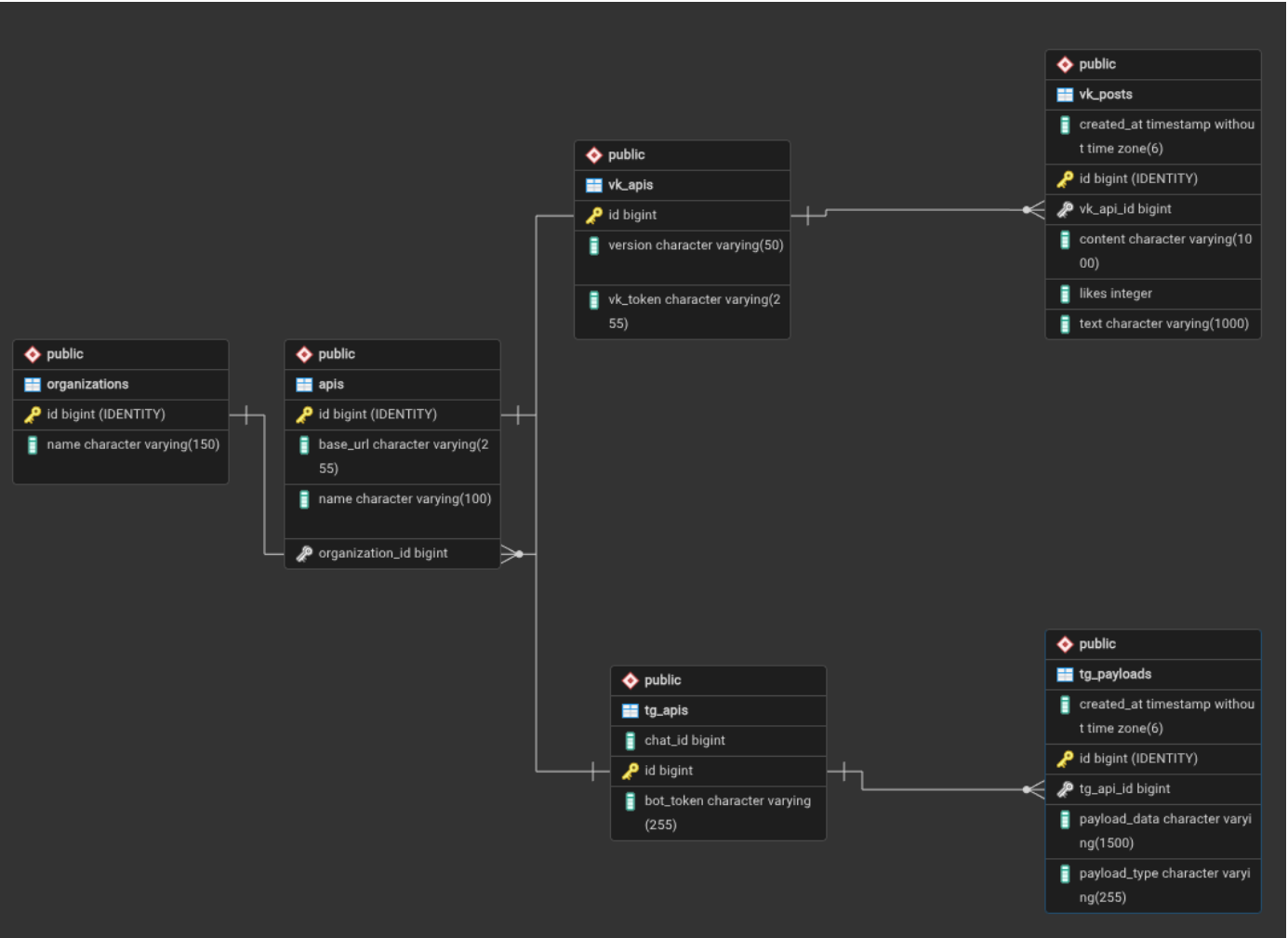


Рисунок 1 – ERD диаграмма

Диаграмма классов

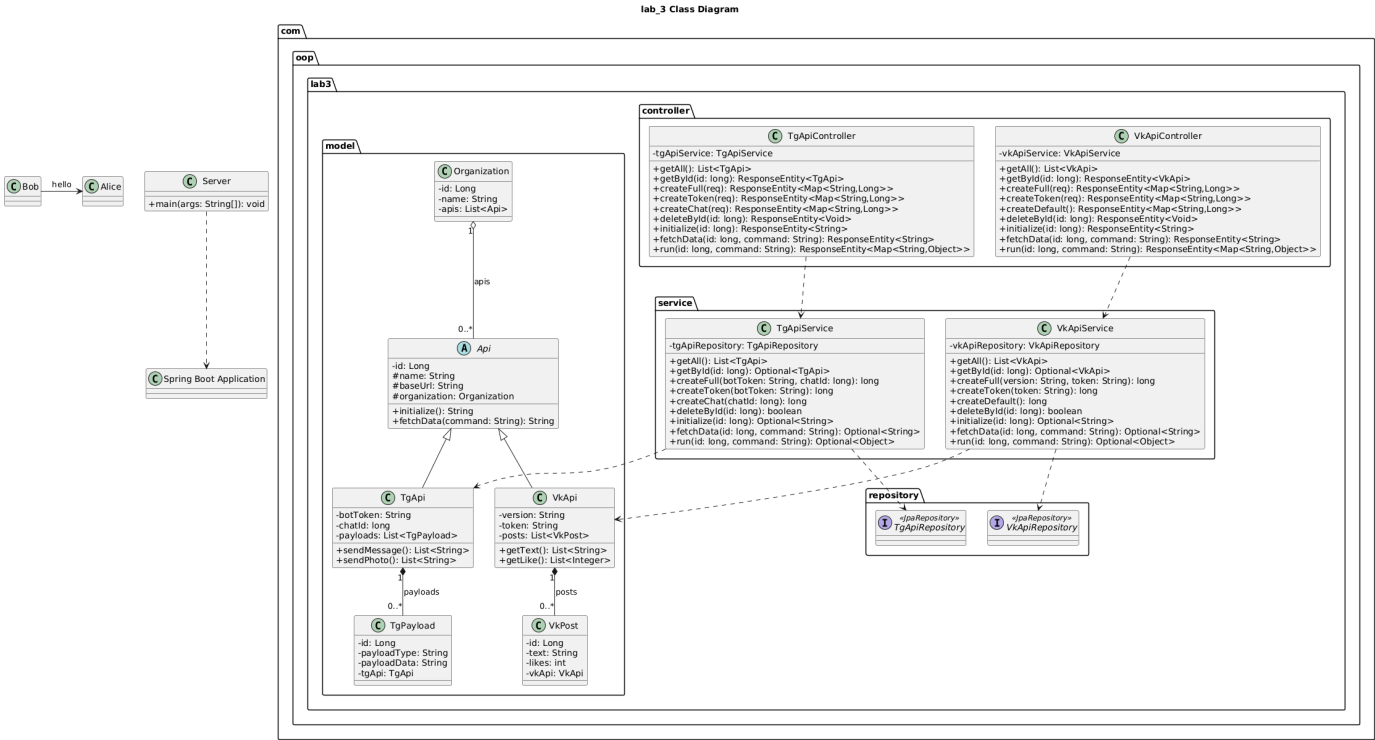


Рисунок 2 – Диаграмма классов

Вывод

Цель лабораторной работы достигнута, а задание выполнено полностью: разработано web-приложение на Java для предметной области VK и Telegram API с хранением данных в PostgreSQL и работой через Hibernate/JPA. CRUD-операции реализованы через `CrudRepository/JpaRepository`, а структура БД включает связанные таблицы и связи не только по наследованию.

В процессе выполнения работы я научилась описывать модель предметной области через JPA-аннотации, настраивать наследование сущностей (`JOINED`), строить связи `one-to-many/many-to-one`, работать с сервисным и контроллерным слоями Spring Boot и вызывать методы предметной области через HTTP API.

Основные сложности возникли при работе с Hibernate: корректная настройка связей между сущностями, требования к конструкторам по умолчанию, а также понимание того, как JPA автоматически формирует таблицы и внешние ключи. После настройки аннотаций и проверки схемы в PostgreSQL эти проблемы были решены.